

Python

Списки. Работа с памятью



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

План занятия

- Изменяемые типы данных
- Примеры вложенных коллекций
- Изменение элементов вложенных списков
- Итерация по вложенным коллекциям
- Оператор del
- Поверхностное и глубокое копирование
- Глубокое копирование



ОСНОВНОЙ БЛОК





Изменяемые типы данных

Изменяемые типы данных



Изменение в месте – позволяет модифицировать содержимое без создания нового объекта.



Ссылочная природа – передача в функцию или переменную создаёт ссылку на объект, а не его копию.



Эффективность – уменьшение нагрузки на память за счёт работы с одним объектом.



Методы изменения – возможность добавлять, удалять и изменять элементы внутри структуры.



Куча

Это область динамической памяти, в которой хранятся все объекты в Python.

Основные аспекты работы с кучей



Выделение памяти – автоматически выполняется интерпретатором Python.



Доступ по ссылке – переменные хранят ссылки, а не сами объекты.



Изменяемые и неизменяемые объекты – неизменяемые (int, str, tuple) создают новые объекты при изменении, изменяемые (list, dict, set) модифицируются на месте.



Сборщик мусора – Python автоматически очищает неиспользуемые объекты.



Функция id

Возвращает уникальный идентификатор объекта, который представляет его местоположение в памяти.

Пример использования id()



```
a = [1, 2, 3]
```

```
b = a
```

```
print(id(a)) # Идентификатор объекта a
```

```
print(id(b)) # Тот же идентификатор
```

```
c = [1, 2, 3]
```

```
print(id(c)) # Другой идентификатор, так как это другой объект
```



ВОПРОСЫ





Как работает присваивание

Присваивание неизменяемых объектов



Когда неизменяемый объект передаётся в другую переменную или функцию, Python копирует по ссылке.

Пример со строкой



```
text1 = "hello"

text2 = text1 # text2 ссылается на тот же объект

text1 + " Python" # Создаётся новый объект

print(text1)

text2 += " world" # Создаётся новый объект "hello world"

print(text1)

print(text2)
```

Присваивание изменяемых объектов



Когда изменяемый объект присваивается другой переменной, копирование происходит **по ссылке**.

Пример со строкой



```
list_a = [1, 2, 3]

list_b = list_a # list_b ссылается на тот же объект, что и list_a

# Добавление нового элемента

list_b.append(4) # Изменение объекта по ссылке

print(list_a)

print(list_b)

# Изменение элемента по индексу

list_b[0] = 'new'

print(list_a)

print(list_b)
```



ВОПРОСЫ





ЗАДАНИЕ

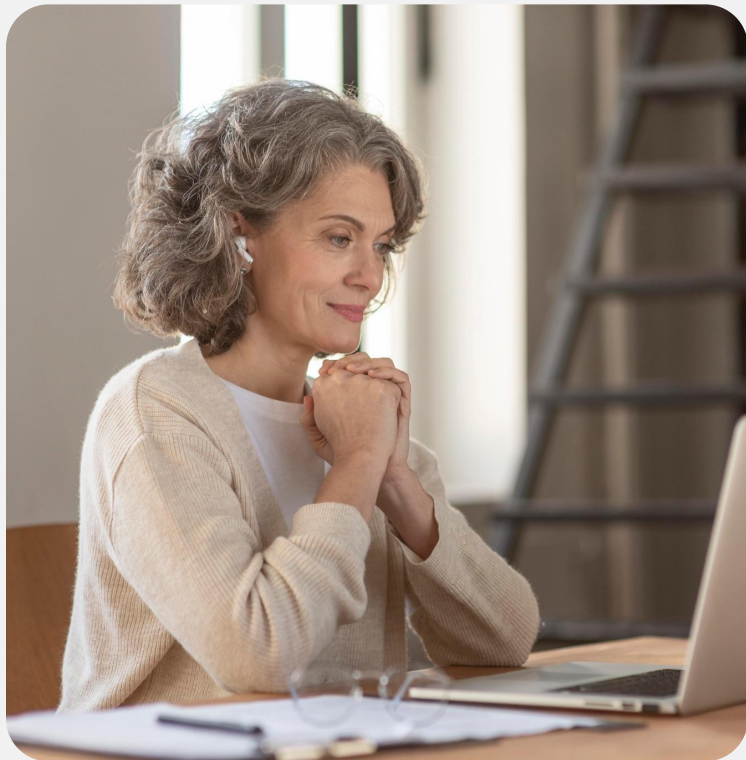




Выберите правильный вариант ответа

Что произойдёт, если две переменные ссылаются на один и тот же изменяемый объект, и его содержимое изменится через одну из переменных?

- a. Содержимое объекта останется неизменным.
- b. Изменится содержимое только одной переменной.
- c. Изменится содержимое обеих переменных, так как они ссылаются на один объект.
- d. Python создаст новую копию объекта для каждой переменной.



Выберите правильный вариант ответа

Что произойдёт, если две переменные ссылаются на один и тот же изменяемый объект, и его содержимое изменится через одну из переменных?

- a. Содержимое объекта останется неизменным.
- b. Изменится содержимое только одной переменной.
- c. Изменится содержимое обеих переменных, так как они ссылаются на один объект.**
- d. Python создаст новую копию объекта для каждой переменной.



Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
a = [10, 20, 30]
```

```
b = a
```

```
b.append(40)
```

```
a.append(30)
```

```
print(a)
```

- a. [10, 20, 30]
- b. [10, 20, 30, 30]
- c. [10, 20, 30, 30, 40]
- d. [10, 20, 30, 40, 30]



Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
a = [10, 20, 30]
```

```
b = a
```

```
b.append(40)
```

```
a.append(30)
```

```
print(a)
```

- a. [10, 20, 30]
- b. [10, 20, 30, 30]
- c. [10, 20, 30, 30, 40]
- d. [10, 20, 30, 40, 30]



Выберите правильный вариант ответа

Что возвращает функция `id()` в Python?

- a. Тип объекта.
- b. Длину объекта.
- c. Уникальный идентификатор объекта в памяти.
- d. Значение объекта.



Выберите правильный вариант ответа

Что возвращает функция `id()` в Python?

- a. Тип объекта.
- b. Длину объекта.
- c. Уникальный идентификатор объекта в памяти.**
- d. Значение объекта.



Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
x = [1, 2, 3]
```

```
y = [1, 2, 3]
```

```
print(id(x) == id(y))
```

- a. True
- b. False
- c. Ошибка
- d. Зависит от типа данных



Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
x = [1, 2, 3]
```

```
y = [1, 2, 3]
```

```
print(id(x) == id(y))
```

- a. True
- b. False**
- c. Ошибка
- d. Зависит от типа данных



ВОПРОСЫ





Вложенные коллекции



Вложенные коллекции

Это коллекции, которые содержат другие коллекции в качестве элементов. Это позволяет строить более сложные структуры данных, такие как многомерные массивы, таблицы и другие.

Примеры вложенных коллекций



Список с числами и списками

```
list_elements = [[1, 2, 3], [4, 5], 6, [7], [8, 9]]
```

```
print(list_elements)
```

Кортеж со списками

```
lists_tuple = ([1, 2], [3, 4], [5, 6])
```

```
print(lists_tuple)
```

Список со списками

```
lists = [[1, 2], [3, 4]]
```

```
print(lists)
```



ВОПРОСЫ





Доступ к вложенным элементам

Доступ к вложенным элементам



Пояснение

Для доступа к элементам вложенного списка используется несколько уровней индексации.

Вложенными элементами

```
list_elements = [[1, 2, 3], [4, 5], 6, [7, [8,  
[9], 10]]]
```

Доступ к вложенным элементам



Первого уровня

```
print(list_elements[0]) # [1, 2, 3]
print(list_elements[1]) # [4, 5]
print(list_elements[2]) # 6
print(list_elements[3]) # [7, [8, [9], 10]]
```

Вложенные списки

```
print(list_elements[0][1]) # 2
```

Доступ к вложенным элементам



Временные переменные

```
first_nested_list = list_elements[0]

second_element = first_nested_list[1]

print(second_element) # 2
```

Глубокая вложенность

```
print(list_elements[3][1][2]) # 10

print(list_elements[3][1][1][0]) # 9
```



ВОПРОСЫ





Доступ к вложенным элементам

Доступ к вложенным элементам



Изменения элементов

```
list_elements = [[1, 2], [3, 4], [5, 6]]
```

Замена элемента в списке

```
list_elements[0][1] = "two"

print(list_elements) # [[1, 'two'], [3, 4], [5, 6]]
```

Доступ к вложенным элементам



Замена вложенного списка

```
list_elements[2] = "new"

print(list_elements) # [[1, 'two'], [3, 4], 'new']
```

Строки внутри списка

```
list_elements[0][1] =
list_elements[0][1].upper()

print(list_elements) # [[1, 'TWO'], [3, 4], 'new']
```




ВОПРОСЫ





Итерация по вложенным коллекциям

Первый уровень вложенности



Пояснение

Простая итерация по верхнему уровню коллекции.

Код

```
for sublist in list_elements:
    print(sublist)
```

Несколько уровней вложенности



Пояснение

Используем вложенные циклы `for`.

Код

```
for sublist in list_elements:
    for item in sublist:
        print(item, end=" ")
```

Итерация с изменением элементов



Пояснение

Изменяем элементы коллекции во время итерации.

Код

```
for i, sublist in enumerate(list_elements):
    for j, item in enumerate(sublist):
        list_elements[i][j] = item + 1

print(list_elements)
```



ВОПРОСЫ





Оператор del

Оператор del



Используется для удаления элементов коллекции по индексу, срезу, а также для удаления целых переменных.

Основные аспекты работы с кучей



Может удалять отдельные элементы, срезы или целые объекты.



Не возвращает значение удалённого элемента.



Освобождает память путём удаления переменных.



Попытка обращения к удалённому объекту вызывает `NameError`.

Синтаксис



```
del list[index] # Удаление одного элемента по индексу
```

```
del list[start:end] # Удаление среза элементов
```

```
del variable # Удаление переменной
```

Примеры



Удаление элемента по индексу

```
numbers = [10, 20, 30, 40]
```

```
del numbers[2]
```

```
print(numbers) # [10, 20, 40]
```

Удаление первого элемента

```
fruits = ["apple", "banana", "cherry"]
```

```
del fruits[0]
```

```
print(fruits) # ['banana', 'cherry']
```

Примеры



Удаление среза

```
numbers = [10, 20, 30, 40, 50]
```

```
del numbers[1:3]
```

```
print(numbers) # [10, 40, 50]
```

Удаление всех элементов (аналог `clear`)

```
numbers = [10, 20, 30, 40]
```

```
del numbers[:]
```

```
print(numbers) # []
```

Примеры



```
# Удаление переменной
```

```
old_numbers = [1, 2, 3]
```

```
new_numbers = old_numbers
```

```
del old_numbers
```

```
print(new_numbers) # [1, 2, 3]
```



ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
nested_list = [10, [20, 30], [40, [50, 60]]]
```

```
nested_list[1][1] = "new"
```

```
print(nested_list)
```

- a. `[10, [20, "new", 30], [40, [50, 60]]]`
- b. `[10, [20, "new"], [40, [50, 60]]]`
- c. `[10, [20, 30], [40, [50, "new"]]]`
- d. Ошибка



Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
nested_list = [10, [20, 30], [40, [50, 60]]]
```

```
nested_list[1][1] = "new"
```

```
print(nested_list)
```

- a. [10, [20, "new", 30], [40, [50, 60]]]
- b. [10, [20, "new"], [40, [50, 60]]]**
- c. [10, [20, 30], [40, [50, "new"]]]
- d. Ошибка



Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

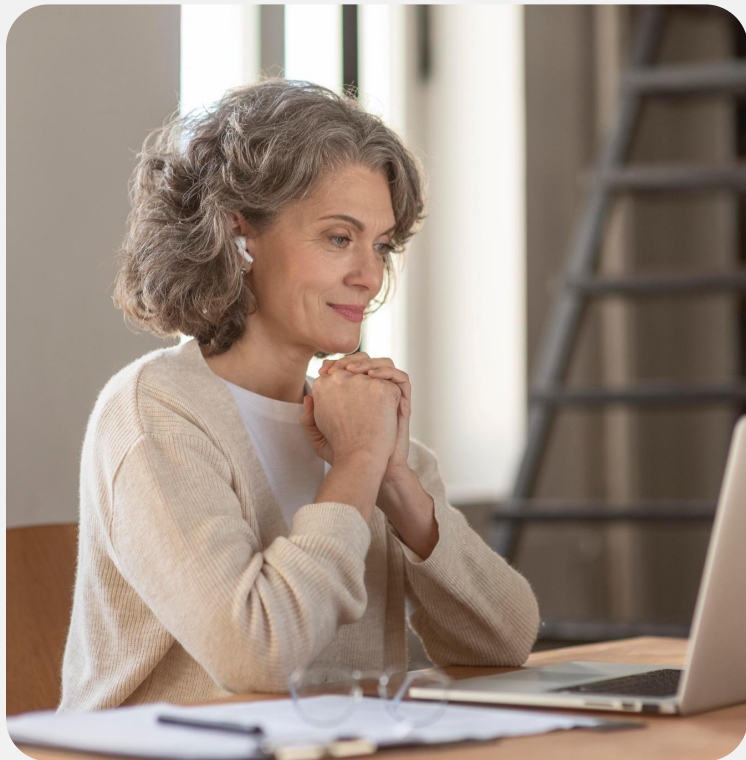
```
a = [1, 2, 3]
```

```
b = a
```

```
del a
```

```
print(b)
```

- a. Ошибка, так как список удалён
- b. `[1, 2, 3]`
- c. `[]`
- d. Переменная `b` тоже будет удалена



Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
a = [1, 2, 3]
```

```
b = a
```

```
del a
```

```
print(b)
```

- a. Ошибка, так как список удалён
- b. [1, 2, 3]**
- c. []
- d. Переменная **b** тоже будет удалена



ВОПРОСЫ





Копирование списка

Копирование списка



При копировании списков важно понимать, что присваивание списка другой переменной не создаёт новую копию. Вместо этого создаётся ссылка на тот же объект.

Пример присваивания



```
original_list = [1, 2, 3]
copied_list = original_list # Это не создаёт новую копию
copied_list[0] = 99
print(original_list) # [99, 2, 3]
```

Важно



При изменении `copied_list` изменяется и `original_list`, так как они ссылаются на один и тот же объект в памяти.



ВОПРОСЫ





**Поверхностное и
глубокое
копирование**

Перегруженный интерфейс и хаотичность

Поверхностное копирование

Подходит для списков без вложенных структур или если вложенные элементы не будут изменяться.

Глубокое копирование

Рекомендуется для списков со вложенными объектами, когда нужно создать независимую копию на всех уровнях.

Поверхностное копирование



Создаёт новый список, копируя элементы из исходного списка, то есть ссылки на них. Если в списке есть вложенные списки или другие изменяемые объекты, то изменение этих вложенных объектов в копии также отразится на исходном списке.

Метод copy()



```
original_list = [1, 2, 3]
copied_list = original_list.copy()
copied_list[0] = 99
print(original_list) # [1, 2, 3]
print(copied_list) # [99, 2, 3]
```

Срез [:]



```
original_list = [1, 2, 3]
copied_list = original_list[:]
copied_list[0] = 99
print(original_list) # [1, 2, 3]
print(copied_list) # [99, 2, 3]
```

Функция list()



```
original_list = [1, 2, 3]
copied_list = list(original_list)
copied_list[0] = 99
print(original_list) # [1, 2, 3]
print(copied_list) # [99, 2, 3]
```

Копирование списка с вложенными объектами



```
original_list = [[1, 2], [3, 4]]  
  
shallow_copy = original_list.copy()  
  
shallow_copy[0][0] = 99  
  
print(original_list)  # [[99, 2], [3, 4]]  
  
print(shallow_copy)  # [[99, 2], [3, 4]]
```




ВОПРОСЫ





Глубокое копирование

Глубокое копирование



Создаёт новый список и полностью копирует все вложенные структуры, создавая независимые копии каждого вложенного объекта. Для этого используется функция `copy.deepcopy()` из модуля `copy`.

Пример глубокого копирования



```
import copy

original_list = [[1, 2], [3, 4]]

deep_copy = copy.deepcopy(original_list)

deep_copy[0][0] = 99

print(original_list)  # [[1, 2], [3, 4]]

print(deep_copy)     # [[99, 2], [3, 4]]
```



ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
original_list = [[1, 2], [3, 4]]
shallow_copy = original_list.copy()
shallow_copy[1][0] = 0
print(original_list)
```

- a. `[[1, 2], [3, 4]]`
- b. `[[1, 2], [0, 4]]`
- c. `[[1, 0], [3, 4]]`
- d. Ошибка



Выберите правильный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
original_list = [[1, 2], [3, 4]]
shallow_copy = original_list.copy()
shallow_copy[1][0] = 0
print(original_list)
```

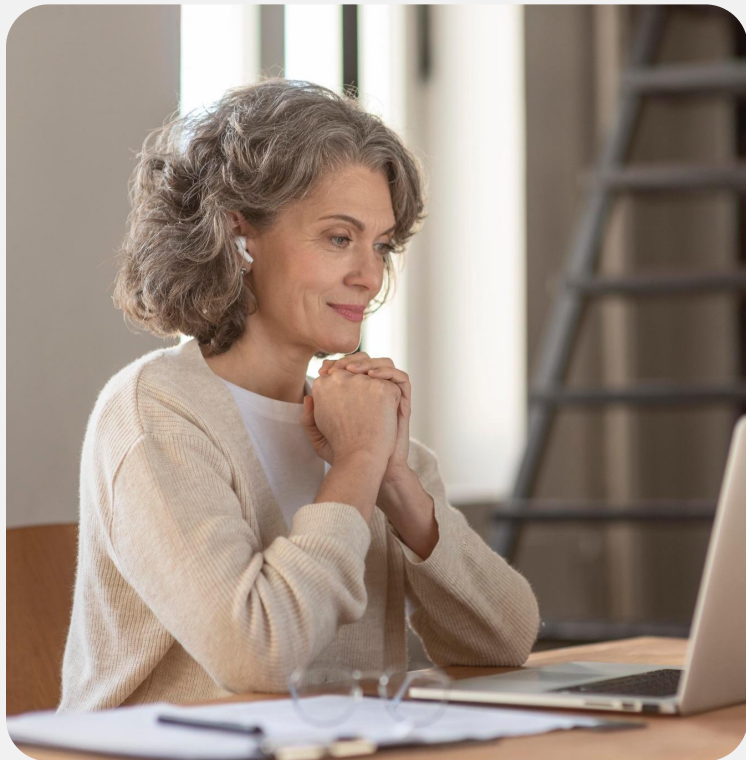
- a. `[[1, 2], [3, 4]]`
- b. `[[1, 2], [0, 4]]`**
- c. `[[1, 0], [3, 4]]`
- d. Ошибка



Выберите правильный вариант ответа

Какой метод используется для создания поверхностной копии списка?

- a. `deepcopy()`
- b. `copy()`
- c. `clone()`
- d. `duplicate()`



Выберите правильный вариант ответа

Какой метод используется для создания поверхностной копии списка?

- a. `deepcopy()`
- b. `copy()`**
- c. `clone()`
- d. `duplicate()`



ВОПРОСЫ





Практические задания





Фильтрация элементов в группах

Напишите программу, которая создаёт копию вложенного списка. Затем в копии необходимо удалить элементы, которые меньше среднего значения всех элементов вложенного списка. Убедитесь, что исходный список остался неизменным.

Данные:

```
nested_list = [[10, 15, 20], [5, 25, 30], [35, 40, 80]]
```

Пример вывода:

Исходный список: [[10, 15, 20], [5, 25, 30], [35, 40, 80]]

Глубокая копия после изменений: [[15, 20], [25, 30], [80]]

Решение



```
import copy

nested_list = [[10, 15, 20], [5, 25, 30], [35, 40, 80]]

deep_copy = copy.deepcopy(nested_list)

for sublist in deep_copy:

    avg = sum(sublist) / len(sublist)

    for i in range(len(sublist) - 1, -1, -1): # Проход с конца, чтобы избежать смещения

        if sublist[i] < avg:

            del sublist[i]

print("Исходный список:", nested_list)

print("Глубокая копия после изменений:", deep_copy)
```



ВОПРОСЫ



Домашнее задание

Одно слово

Напишите программу, которая обрабатывает список из строк и удаляет все строки, содержащие более одного слова, а также преобразует оставшиеся строки к нижнему регистру.

Данные:

```
text_list = ["Hello", "Python Programming", "World", "Advanced Topics", "Simple"]
```

Пример вывода:

Обработанный список: ['hello', 'world', 'simple']

Домашнее задание

Обновление цен товаров

Дан список товаров с ценами. Программа должна применить скидку к каждому товару и добавить в список элемент с новой ценой. В конце вывести таблицу с новой ценой.

Данные:

```
products = [{"Laptop", 1200}, {"Mouse", 25}, {"Keyboard", 75}, {"Monitor", 200}]
```

Пример вывода:

Введите скидку (в процентах): 17

Товар	Старая цена	Новая цена
Laptop	1200.00\$	996.00\$
Mouse	25.00\$	20.75\$
Keyboard	75.00\$	62.25\$
Monitor	200.00\$	166.00\$

Заключение

